

What best-practice workflow costs

THE ANSWER, IN ONE LINE

About 9 to 14 engineer-months to bring every product to the shape we want. But the first **2 to 3 engineer-months** buy almost all of the value, and move no workflow at all.

Rp265M to Rp700M for the whole programme; Rp60M to Rp160M for the part that is worth doing whatever else we decide. Compare that with 30 to 57 engineer-months for either platform migration.

Question asked What would it cost to bring Bravo's workflows up to best practice?

Evidence ms-bpm working tree · production BPM Postgres, 90 days to 2026-09-09

As at 2026-09-11

What most people believe about Bravo’s workflows is not what production shows

“One giant workflow handles all products” describes a shared spine that carries **5.5% of applications and exactly one product**. What is actually expensive today is the *older, one-workflow-per-product* design — which is, almost exactly, the design usually proposed as the fix.

ROOT WORKFLOW	STARTED, 90 DAYS	SHARE	GENERATION
NDF2W	248,682	73.4%	older monolith – 174 nodes, 359 commits
NDF4W	59,909	17.7%	older monolith – 223 nodes, 392 commits
NDF4W_RO	11,458	3.4%	older variant
Unified_Process_Main_Workflow	18,809	5.5%	the shared spine – 43 nodes, no user tasks
Of the 18,806 applications on the shared spine, every one is DF4W, plus a 4-application NDF4W company pilot. DF2W is fully configured with no volume because it has not been released: it is in UAT, with a penetration test running against the LOS.			

THE GIANT WORKFLOWS ARE THE PER-PRODUCT ONES

The shared spine is **43 top-level nodes across 33 files**. NDF4W is 223 and NDF2W is 174. The older generation is the one-workflow-per-product design, and it is where the complexity built up: **156 different delegate beans** across the older roots, against **69** for the five products on the newer one.

A GRADUAL REPLACEMENT HAS ALREADY STARTED BY ITSELF

The shared spine has run DF4W in production **for months, steadily, at 5.5%**. Nobody planned it as a migration, but it proves the shape we want can carry real volume. The programme below is mostly about **doing that on purpose**.

Product differences live in four places, and the diagram shows none of them

This is what the effort below is buying out — not the size of any diagram. Nothing in the repository shows what a given product actually runs.

MECHANISM	WHERE	MEASURED TODAY	WHY IT IS A PROBLEM
Gateway conditions on the product	<code>unified-main-workflow</code> , <code>-underwriting</code> , <code>-pd-model</code>	3 files carry <code>applicationWorkflowSelectorType</code> ; 9 different feature flags are read from BPMN via <code>environment.getProperty</code>	Product routing and feature flags are mixed together inside the process XML . The diagram reads as one flow but runs as several
Tasks that do nothing, switched off by config	<code>BaseActivity.isNeedToProceed</code> and the <code>workflow_*</code> tables	55 classes gated this way; 64 workflow-config migrations since March 2024	A skipped task records itself as completed , so Camunda history, Cockpit and the BPMN all show steps that did nothing. The only way to know a product’s real path is a five-table join
Product branches inside Java	<code>activity/unified/**</code>	19 of 68 classes ; 42 call sites — <code>isDF4W()</code> 21, <code>isDF2W()</code> 7, <code>isNDF2W()</code> 7, <code>isNDF4W()</code> 5, <code>isDF2WSharia()</code> 2	The same step behaves differently for different products, with no trace of it in the BPMN or the config . This is the layer that grows quietly
Feature flags per product	<code>application.yaml</code>	<code>featDF2W</code> , <code>featDF2WSharia</code> , four <code>featBypassScoringProcess*</code> — among 726 feat* keys in total	A product can be half-enabled in one environment , and the BPMN has no way to show that

THE STEPS THAT DO NOTHING ARE MEASURED, NOT HYPOTHETICAL

In `Unified_Process_KYC_Check`, Address Verification, Phone Checking and Shopee Score each ran **4,822 times in 7 days**, with **86 to 89% of those runs finishing in under 50 milliseconds** — faster than a network call can complete. They are switched on for DF4W only. For every other product they log “Skipping” and return, while history records a completed step. The real calls in the same process take **898 to 1,874 milliseconds**.

Eight principles define best practice. Bravo meets one and a half.

PRINCIPLE	WHERE BRAVO IS TODAY	SEVERITY
A The diagram is the truth: structure in the diagram, data in config	Structure is encoded as <code>is_active</code> rows for steps that do nothing. 55 classes are gated this way. You cannot work out the real path from the diagram	High
B Each product owns a thin spine of its own	One shared spine serves all five products, and the product is chosen by gateways and a selector variable. No product owns a readable spine	High
C Shared sub-workflows, with no product logic inside them	19 of 68 shared steps branch on <code>application.isXxx()</code> , across 42 call sites	High
D Split a sub-workflow only when the structure genuinely differs	Done correctly once – <code>...Underwriting_Regular</code> . Everywhere else, differences are handled by switching steps off in config. The good example is the exception	Medium
E Keep the product out of shared code, behind stable contracts	Config, Java, gateways and flags all carry the product: four separate places to change to alter one product’s flow	High
F The engine should have only the rights it needs, and deploy per product	<code>/camunda</code> is set to <code>permitAll</code> . <code>/engine-rest</code> gives full control of the engine to anyone holding the shared key – this is how the injected code got in . And there is one deployment unit for all products	High (security)
G Replace gradually, never all at once	Happening – but by accident , stuck at 5.5%, with no migration plan. The older workflows still carry about 94% of volume and the whole retail book	Medium
H Dead and duplicated things are deleted	Four dead <code>setting.workflow.map</code> keys; one mock deployment nothing refers to; 10 named 2W/4W delegate pairs doing the same job twice	Low-Med

Bravo has the right building blocks, and one correctly-split sub-workflow. What it does wrong is express product differences in three places the diagram cannot show.

So no product has a readable spine of its own that can be deployed independently — and the older monoliths carrying 94% of volume were never migrated at all.

A spine per product, shared sub-workflows, and differences out in the open

KEEP — WHAT THE NEWER DESIGN GOT RIGHT

- **The top-level workflows contain no human tasks.** Human work sits only in the sub-workflows. Production confirms it: the top-level workflows recorded **no user tasks at all** in 90 days
- **Each domain step exists once.** Adding DF2W Sharia in 2026 touched the spine in one commit plus some migrations, instead of adding a fourth monolith
- **Escalation, cancellation and rejection are handled in one place**, as boundary events on the spine, rather than redrawn for every product
- **The right kind of split already exists:**
`Unified_Process_Workflow_Underwriting_Regular` is a DF2W-family version called from the shared underwriting workflow. The team knows the pattern. It is just not applied consistently

CHANGE — FIVE RULES, IN ORDER OF PAYOFF

- **1 • One thin spine per product** (about 40 nodes). It is the single source of truth for the order of stages and which sub-workflow each stage calls. The spine *does* the selecting, so the gateways go away
- **2 • No silent skipping.** A task that does not apply to a product should not be on that product's path at all. At the very least, a missing config row must **fail loudly**
- **3 • Take `application.isXxx()` out of the 19 shared steps.** Product behaviour belongs in the spine or in a product-specific sub-workflow, never in a shared bean
- **4 • Split a sub-workflow only when the structure differs** — a different order of steps, different human roles, different partners. A different threshold does not count
- **5 • Publish the real flow for each product**, generated on every build and diffed in CI

CAMUNDA 7 MAKES THE MIDDLE PATH CHEAP

A call activity's `calledElement` can be **an expression**. So a product spine can send most products to the shared `survey` sub-workflow and one product to `survey_sharia`, **with no gateway and no flag**. Five spines of about 40 nodes each cost far less than the 25 per-product sub-workflows a full split would need — and duplicating everything is exactly how `NDF4W` and `NDF2W` drifted apart in the first place.

Where these estimates come from — and how much to trust each phase

HOW THE EFFORT IS COUNTED

Engineer-days of a competent engineer **who already knows ms-bpm**, including that engineer’s own testing but **not UAT run by business users**. 21 working days to the engineer-month. Rupiah figures use the assumption applied throughout this pack: **Rp30–50M fully loaded per engineer-month** — substitute BFT’s own rate card. Ranges are **low to high**. They **are not confidence intervals**.

Unlike the end-of-support numbers, these did not come from a team memo. We derived them here by counting what is there — delegates, branch sites, gated classes, user tasks, production volumes — and the squads who would do the work should re-estimate them.

THE ONE MEASUREMENT THAT WOULD FIRM UP PHASES 3 AND 4

Of NDF2W’s **67 delegate beans, exactly one** (`checkUnderwritingNeededUnifiedActivity`) is also used by the shared sub-workflows. **So migrating is not a matter of pointing things somewhere else. It means working out, bean by bean, whether the shared sub-workflow already does that job.**

Mapping the delegates — **5 to 8 days** — would turn the widest range in this deck into a real number. It is the direct equivalent of the LORA coverage-gap inventory.

HOW CONFIDENT WE ARE, BY PHASE

PHASE	CONFIDENCE	WHY
● P1 VISIBILITY	High	The queries exist (§8.4) and so do the parse scripts. It is tooling work of a known shape
● P2 PROVE ON DF4W	Medium-high	You can list the scope exactly: one spine, 3 gateways, 21 <code>isDF4W()</code> sites, and a known set of gated KYC and RAC steps
● P3 NDF2W	Low-medium	It depends on how much of NDF2W’s behaviour the shared sub-workflows already cover, and nobody has measured that . The widest range here
● P4 THE REST	Low	Scaled up from P3 plus what we counted. Sharia is a separate deployment that nobody has scoped

WHAT EVERY NUMBER HERE LEAVES OUT

UAT run by the business, product-owner workshop time, rewriting domain logic (how survey and underwriting actually *behave* is out of scope), and any change to the LORA or Temporal track.

See it, and stop it getting worse. No workflow moves.

WORK ITEM	WHAT IT INVOLVES	ENGINEER-DAYS	DONE WHEN
Publish the real flow for each product	Join <code>workflow_master_config_detail</code> , <code>selector_order</code> and <code>selector_type</code> , and lay the result over the BPMN. One diagram per product, regenerated on every build. The §8.4 query and the parse scripts already exist	8-13	Every live product has a current flow diagram its owner can read
Stop new hidden differences appearing	A pull-request check that fails on a new <code>application.isXxx()</code> in <code>activity/unified/**</code> , or a new config-gated class with no documentation entry	2-4	CI blocks both, and the count of branch sites can only fall from 42
Secure the engine	Require authentication and restrict the network on <code>/camunda</code> and <code>/engine-rest</code> ; rotate <code>INTERNAL_SERVICE_KEY</code> ; register a <code>ProcessEngineAuthenticationFilter</code> ; delete the 61 injected definitions	3-5 already in Decision A	Injected definitions deleted, verified in production and in Sharia
Agree the target	A one-page ADR – a spine per product, shared sub-workflows, split only on structure – signed off by the product owners	2-3	ADR merged
Cheap cleanup	Delete the four dead <code>setting.workflow.map</code> keys and remove <code>Process_NDF4W_Scoring_1_Mock_Ro</code>	0.5-1	Every map key points to a real process
Phase 1 total – 1-2 engineers, about 1 month		15-26 days about 0.8-1.3 engineer-months	Rp 25M – 65M

WHAT PHASE 1 DELIBERATELY DOES NOT DO

No workflow moves, no BPMN is restructured, and no product migrates. Phase 1 changes **what can be seen, and what can be added**. It makes each product’s real path visible and stops the number of hidden branches rising. Every structural change waits for Phase 2, behind a signed-off ADR.

THIS PHASE IS WORTH DOING WHATEVER DECISION B TURNS OUT TO BE

The per-product flow diagram is **exactly what the LORA gap inventory needs as input** — “every Bravo delegate, status, assignment rule and approval path mapped to a LORA ProcessStep”. If BFI migrates to LORA, this work is a **prerequisite, not money wasted**. If it moves to Temporal, a faithful port of a model nobody can read carries the problem across — this makes the port smaller. And if Bravo stays, it is the foundation for everything below.

Prove the pattern on DF4W — the only product already on the spine

This is the lowest-risk pilot available: 5.5% of volume, one product, already running in production on the shared spine. If the target shape does not work here, it does not work anywhere.

WORK ITEM	SCOPE, AS COUNTED	ENGINEER-DAYS	DONE WHEN
Build <code>spine_df4w</code>	A thin runnable spine naming DF4W’s stages and calling each sub-workflow by <code>calledElement</code> , with new DF4W volume routed to it behind a flag	5-8	New DF4W loans run on <code>spine_df4w</code> , and the shared spine no longer receives DF4W
Remove DF4W’s hidden differences	21 <code>isDF4W()</code> call sites in shared beans, plus the 3 <code>applicationWorkflowSelectorType</code> gateways on the DF4W path	6-10	No product gateways and no <code>isDF4W()</code> anywhere on the DF4W path
Get rid of the tasks that do nothing	Turn the gated KYC and RAC no-ops into explicit choices in the diagram – a gateway where the choice depends on data, a split where it depends on the product	8-12	No sub-50ms do-nothing service tasks in DF4W history for the gated KYC steps
Lock in the visibility	The per-product flow diagram is generated and diffed in CI , and each sub-workflow gets contract tests	4-6	A change to a shared sub-workflow that alters DF4W’s path shows up as a diagram diff in review
Route it, run it, and check it matches	Gradual routing, a rollback path, and checking latency and errors match the shared spine	4-6	It matched for a full soak period, at DF4W’s roughly 6,800 applications a month
Phase 2 total – 1-2 engineers, about 1 to 1.5 months		27-42 days about 1.3-2.0 engineer-months	Rp 40M – 100M

WHAT PHASE 2 SETTLES THAT NO DOCUMENT CAN

Whether one thin spine plus shared sub-workflows really can carry a product without the config gating — and **what one product actually costs to move**. Phases 3 and 4 should be re-estimated against what Phase 2 really took, rather than the ranges on the next two slides.

NDF2W is both the prize and the expense — 248,682 applications a quarter

WORK ITEM	ENGINEER-DAYS	SCOPE, AS COUNTED
Map NDF2W’s delegates onto shared sub-workflows, and build <code>spine_ndf2w</code>	10–15	67 delegate beans, 174 top-level nodes, 45 embedded sub-processes
Extend the shared sub-workflows to cover how NDF2W behaves	25–40	66 of the 67 beans have no equivalent on the newer side. This is the single widest range in the programme
Move the human work	8–12	20 user tasks, including the User Survey Task with 185,574 instances. Who each one goes to is decided in Java, because <code>candidateGroups</code> is empty on every task
Gradual routing, running both, checking they match, and a runbook	10–15	The old <code>NDF2W</code> stays live for loans already running, and for rollback
Regression testing and UAT support	8–12	Engineer time only. UAT run by the business is excluded
Phase 3 total – 2 engineers, about 3 to 4 months	61–94 days about 3.0–4.5 engineer-months	Rp 90M – 225M

1 of 69

NEWER DELEGATE BEANS ALSO USED BY AN OLDER ROOT. THIS IS WHY MIGRATING IS NOT JUST RE-POINTING THINGS.

≈25%

THE GATE TO PASS: SHARE OF NEW NDF2W STARTS ON THE NEW SPINE, MATCHING ON ERRORS AND LATENCY

WHY THIS NUMBER IS HONEST BUT WIDE

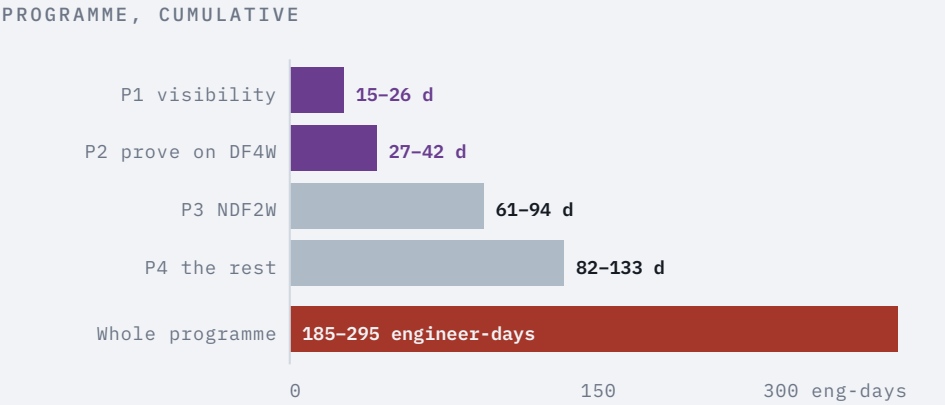
The two generations share **almost no code**. Every NDF2W step has to be classified as one of three things: *the shared sub-workflow already does this*, *the shared sub-workflow needs extending*, or *NDF2W genuinely differs and needs its own version*. Nobody has done that classification. It is the 5-to-8-day mapping on slide 6, and it is worth doing **before** committing to this phase.

THE PAYOFF IS PROPORTIONAL

NDF2W on its own is **73% of Bravo’s originations**. Moving it is what turns this programme from a governance improvement into a real reduction in how much there is to maintain — which is what 359 commits on a single file represents.

The rest of the estate, and what the whole thing costs

PHASE 4 WORK ITEM	ENGINEER-DAYS	SCOPE
Move NDF4W and NDF4W_RO onto spines	50-80	68 + 14 delegate beans, 223 top-level nodes, 21 user tasks. RO reuses NDF4W's sub-workflows – 17 of its 19 steps are identical
Sharia's separate deployment	15-25	Its own engine and its own deployment unit, 12 delegates, never scoped
Merge the duplicated 2W and 4W beans	12-20	10 named pairs – createCif and createCif2w, preFatalRac and ...2w, addressVerification and ...2W, and seven more – behind one shared implementation
Launch DF2W on spine_df2w	5-8	Only the extra work – it is configured and in UAT, not dead config
Phase 4 total – 2 engineers, ~4-6 months elapsed	82-133 days ≈4.0-6.3 eng-months	Rp 120M – 315M



THE STAFFING ASSUMED BEHIND THOSE ELAPSED TIMES

Phases 1 and 2 assume **1-2 engineers**; Phases 3 and 4 assume **2 engineers**, working alongside normal delivery. On that basis the programme runs for roughly **12-18 months**. With three engineers on Phases 3 and 4 it compresses towards 9-12 months, without changing the engineer-day totals.

PROGRAMME TOTAL

185-295 engineer-days ≈ 9-14 engineer-months ≈ Rp265M-700M

Compare that with 30-57 engineer-months for either platform migration, and 18-33 engineer-days for the end-of-support fix that has to happen first.

How this compares with everything else competing for the same engineers

PROGRAMME	EFFORT	INDICATIVE RP	WHAT IT BUYS	DEPENDS ON?
Decision A – the end-of-support fix (fork + Spring Boot 4 + the code-execution hole)	18–33 eng-days	Rp45–125M	A supported engine and a supported Spring Boot. Closes a live path for running code on our servers	Nothing do it first
● WORKFLOW P1 ● P2 – see it, freeze it, prove it on DF4W	42–68 eng-days ≈2.0–3.2 eng-months	Rp60–160M	A flow per product its owner can read, CI that stops the hiding, and the target shape proven on live volume	Nothing useful under every option
● WORKFLOW P3 ● P4 – move the monoliths	143–227 eng-days ≈6.8–10.8 eng-months	Rp205–540M	Every product on its own readable spine, deployable independently, and the duplication gone	How long Bravo has and Decision B
Option 2 – move Bravo to Temporal	31–57 eng-months	Rp930M–2.85B	Removes the workflow-engine vendor for good, but keeps the architecture exactly as it is	Decision B
Option 3 – move Bravo to LORA	30–57 eng-months	Rp900M–2.85B	One platform. You can add an automated step without editing a workflow	Decision B

The whole workflow programme costs about a quarter of either migration, and it is the only one of the three that improves Bravo’s design without changing platform.

Its first half is worth funding even if BFI eventually leaves Bravo: a flow map per product is what the LORA gap inventory needs first, and a readable model makes a Temporal port smaller rather than a faithful port of the same confusion.

What leaving things as they are costs, measured rather than asserted

EVERY CHANGE CAN BREAK EVERYTHING

Commit messages on `unified-main-workflow.bpmn` name **DF4W, DF2W, Sharia, RO, NDF4W and Company**. Every one of those redeployed the spine that **all five products** run on. On the survey sub-workflow, DF4W work changed the orchestration NDF2W also uses. One deployment unit means every product shares the risk of every edit.

A TEST MATRIX NOBODY CAN WRITE DOWN

Whether one BPMN file is correct depends on **products, times config rows, times flags**. The **64 workflow-config migrations** include repeated pairs that correct each other — `pd-model-df4w-set-active`, then `underwriting-return-set-inactive-except-df4w`. That is what a hidden matrix looks like in practice: **about 2 corrective migrations a month** since March 2024.

THINGS GO QUIETLY WRONG INSTEAD OF LOUDLY FAILING

Because `BaseActivity` returns quietly, **a missing config row does not fail a deployment or a process. It just removes a check from a loan product** — and history still shows the step completing. There is no alert for this kind of defect today.

NOTHING YOU CAN HAND A PRODUCT OWNER

You cannot show a stakeholder a diagram of their product. The diagram is shared, and the differences live in SQL and Java. **Who owns which product is not recorded anywhere in the model** — `candidateGroups` is empty on every user task, and assignment happens in code.

THE DUPLICATION KEEPS GROWING

The older generation holds **156 different delegate beans**, and the two big monoliths share only **20** of them. Most of the rest is the same domain step under a slightly different name. **Duplicating everything per product is how NDF4W and NDF2W drifted apart**, and it keeps happening wherever a product needs one step done differently.

WHAT IS *NOT* A REASON TO DO THIS

Engine cost. Bravo's workflow tier is the **cheapest of the three platform options, at about Rp58M a month**. This programme is about being able to read the system, limiting what one change can break, and correctness. **It is not about savings, and it will not pay for itself in infrastructure.**

What could make these numbers wrong, and what has to be true before we start

RISK	EFFECT ON THE ESTIMATE	WHAT TO DO ABOUT IT
The shared sub-workflows cover less of NDF2W than assumed	This is exactly why Phase 3 has the widest range. A bad case pushes it past 94 days	Do the 5-to-8-day delegate mapping first , and re-estimate P3 and P4 against what Phase 2 actually took
There is no safety net at the engine layer	Every phase is riskier than its day count suggests	Of <code>ms-bpm</code> 's 1,457 test files, 4 run a Camunda process. The parity test suite in Decision A is a prerequisite here too, not a nice-to-have
Loans already running during a cutover	Adds calendar time, not effort, if you plan for it. Adds both if you do not	Replace gradually by design – the old roots stay deployed for loans in progress and for rollback
Decision B picks LORA halfway through	Phases 3 and 4 become wasted money	Do not start P3 until a Bravo horizon is published. P1 and P2 are useful either way
You cannot get the product owners together to sign off the target	Blocks the ADR, which blocks P2	Two squads and three consoles already own these products. The boundary exists; use it

ONE HARD PREREQUISITE

Securing the engine is **not optional, and it cannot be left until later**. `/camunda` is set to `permitAll`, and `/engine-rest` gives full control of the engine to anyone holding the shared key. That is how **61 hostile process definitions** reached production. **Nothing else on this plan should start while an outsider can deploy process definitions into the engine we are restructuring.**

EXPLICITLY OUT OF SCOPE

Rewriting survey or underwriting **domain logic**; migrating Sharia's separate deployment beyond the spine work costed in Phase 4; and any change to the LORA or Temporal track. This plan is only about **the shape of Bravo's workflows**, and about bringing product differences out into the open.

Fund the first half now, and hold the second half until we know how long Bravo has

1	Approve Phases 1 and 2 as spending that does not depend on anything else. Visibility, a CI block on new hidden differences, and the target shape proven on DF4W — the product already running on the spine. No older workflow moves, and the output is needed first whichever way Decision B goes.	42-68 ENG-DAYS ABOUT 2-3 ENG-MONTHS RP60-160M
2	Commission the 5-to-8-day delegate mapping before deciding on Phase 3. It turns the widest range in this deck — how much of NDF2W the shared sub-workflows already cover — into a real number, and it is the direct equivalent of the LORA coverage-gap inventory.	5-8 DAYS DO IT INSIDE P1
3	Hold Phases 3 and 4 until the Bravo horizon is published. Moving 94% of volume onto product spines is worth 143 to 227 engineer-days only if Bravo has years left. That is the same input Decision B needs, and it is a business call.	143-227 ENG-DAYS RP205-540M GATED
4	Accept the monthly metrics below as how this gets reported — and re-baseline Phases 3 and 4 against what Phase 2 actually took, rather than the ranges in this deck.	MONTHLY

PROGRAMME METRIC, REPORTED MONTHLY		TODAY	TARGET
Share of new applications running on an explicit product spine		0 — DF4W on the shared spine does not count	all live products
application.isXxx() call sites in activity/unified/**		42, across 19 of 68 classes	0
Do-nothing service tasks run per day because config switched them off		heavy — 89% of gated KYC steps finish in under 50ms	near zero
Different delegate beans across the older products		156	falling
Products with a current spine diagram their owner can read		0	all live products